Learnova: Rebuilding Lecture Slide Decks into Visually Structured, Interactive Presentations

Kawaljeet Singh Bharaj, Jeevesh Bharat Bodhani, Rishit Pankajkumar Lahoti, Swayam Devendra Panchal
Department of Artificial Intelligence and Data Science
Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
panchalswayam35@gmail.com

***Abstract*—Most lecture slides that students receive are exported PowerPoint files or PDFs made of dense bullet lists, pictures of equations and diagrams with no structure a program can use. Tools that generate presentations from a prompt do not help much here: they start from scratch, paraphrase the teacher's content and cannot be checked against the source. We describe Learnova, a system that takes an existing deck (PPTX, slide PDF, Word document or typed notes) and rebuilds it as an editable PowerPoint file and an interactive web deck while keeping every point the teacher wrote. The system has four parts: a page-by-page reader that recovers titles, bullet nesting, tables and figures from slide PDFs, including slides that are only pictures; a rule-based Content Decision Engine that chooses a visual form for each slide (process, timeline, comparison table, definition, chart and others) with a stated reason; a layout stage that measures text glyph by glyph so that no word is ever broken across lines; and an illustration stage that draws a diagram for every topic slide, written by a language model when an API key is available and drawn by offline rules when it is not. On three held-out gold sets (101 slides) the decision engine picked an acceptable visual for 96.8–97.4% of slides in under one millisecond per slide. On six real course decks (36 to 216 pages) the rebuilt decks kept 98–100% of numeric and named facts, and the layout audit flagged at most 4% of slides, none of them for broken words.**

***Index Terms*—presentation generation, document layout analysis, educational technology, slide redesign, information visualisation, large language models**

I. Introduction

A typical engineering course in our college is taught from slide decks that have been reused for years. When we collected decks from our own semester—natural language processing, machine learning and image processing—we found the same pattern in all of them: long bullet lists, equations pasted as screenshots, diagrams copied from textbooks, and many slides with no title at all. Students read these decks before examinations, and they are not easy to read.

Recent tools such as Gamma [1], Copilot in PowerPoint [2], Gemini in Google Slides [3] and Canva Magic Design [4] make attractive decks from a short prompt. They are designed for creating new presentations rather than repairing old ones. When a lecture deck is pasted into them, the content is rewritten, items of a list are silently dropped, and the result cannot be compared with the original. For a study aid that is a serious problem: the student cannot tell which point went missing. Abstractive summarisers are known to make exactly these errors [5], [6].

Learnova takes the opposite position. The teacher's content is treated as fixed, and the system only changes how it is shown. Our contributions are:

- a reader that turns a slide deck exported to PDF (and, through a PDF rendering, a PPTX file) back into structured slides, including OCR for picture-only slides and recovery of wrapped titles and split words;
- a deterministic Content Decision Engine that maps each slide to one of more than twenty visual families and returns the data that visual needs together with a reason;
- a glyph-measured text-fitting method and a layout audit that guarantee that words are never split across lines;
- a per-slide illustration stage with two fallbacks: a diagram written as SVG by a language model under strict validation, and an offline scene illustrator;
- an evaluation on gold sets and on six real course decks, with all numbers reproducible from the repository.

II. Related Work

Document-to-slide generation. PPSGen [7] learned sentence importance to build slides from papers; DOC2PPT [8] and D2S [9] generate slides from scientific documents with neural summarisation and figure selection. These systems target papers, not existing decks, and they summarise rather than preserve content.

Commercial deck generators. Gamma, Copilot, Gemini and Canva generate an outline first and then fill a template. In our trials they produced good-looking slides but changed wording, merged list items and could not keep the original figures of an uploaded lecture.

Automatic visual design. Mackinlay's APT [10] chose graphical encodings from data characteristics, and Draco [11] encodes visualisation design knowledge as constraints. Our decision engine follows the same idea—explicit, inspectable rules—but applies it to sentences rather than data tables.

Slide design guidance. Assertion–evidence slides, which use a sentence headline supported by a visual, improve comprehension over bullet lists [12], [13]. Mayer's multimedia principles [14] and cognitive load theory [15] motivate keeping related text and pictures together and limiting the number of new items on a slide. Tufte's critique of bullet outlines [16] is another source of the problem we address.

Line breaking. Knuth and Plass [17] treated paragraph layout as an optimisation over break points. We need a much

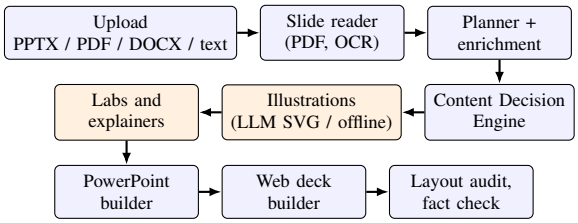


Fig. 1. Learnova pipeline. Orange stages use a language model when a key is set and fall back to offline rules otherwise.

simpler guarantee—no word wider than its box—but in many small boxes at once.

III. System Overview

Fig. 1 shows the pipeline. An upload is converted into a Markdown intermediate representation together with per-slide metadata (figures, page previews, OCR text, colour palette of the source). The planner splits overloaded slides and merges very short ones; the enrichment stage adds an agenda, checkpoint quizzes, key terms and a glossary; the decision engine picks a visual for each slide; the illustration and lab stages prepare pictures and interactive pages; and two builders write the PowerPoint file (python-pptx, native editable shapes) and the web deck (Reveal.js [18]). A FastAPI service runs jobs in the background and a React front end shows progress, previews and exports.

IV. Reading Slide Decks

A slide PDF already contains one slide per page, but its structure is implicit. The reader works on text lines from PyMuPDF, each with a bounding box (x_0, y_0, x_1, y_1), a font size s and a font name.

Furniture removal. A line in the top 12% or bottom 14% of a page is treated as page furniture (course name, author, slide number) when the same normalised text appears in that band on at least

$$m = \max\bigl(3,\ \lfloor 0.3\,n \rfloor\bigr) \tag{1}$$

of the n pages. Page counters of the form "k/n" and the chrome of decks exported by slide tools are removed as well. Repeated pictures are handled in the same way, but we count only images actually drawn on a page: exporters such as LibreOffice place every image of the deck in one shared resource dictionary, and counting resources made every figure look like a logo.

Titles. The title is the largest line in the top third of the page if its size is at least 1.18 times the body size $\bar{s}$ (or it is bold and near the top). A title that wraps is rejoined when the next line has the same size, starts at the same x and its vertical gap g satisfies $-0.5\,s < g < 0.9\,s$; the negative lower bound is needed because line boxes from some exporters overlap.

Bullet levels. Bullet glyphs (including symbol-font characters) are matched to the text line on their baseline. The x positions of the glyphs are clustered with tolerance $\tau = 9$ pt into anchors $a_1 < a_2 < \dots$, and an item's level is the index of its nearest anchor:

$$\ell(x) = \arg\min_{k} \, |x - a_k|. \tag{2}$$

A line with no glyph continues the previous item when it starts within 12 pt of the item's text, follows it vertically and the previous item has not ended a sentence; a lower-case line is accepted with a looser 48 pt tolerance.

Glued titles. OCR and some exporters drop spaces in capitalised titles ("CONSTRUCTINGTHETABLE"). We segment such a token t of length L by dynamic programming over a vocabulary V built from the deck's own words and a dictionary:

$$B(j) = \max_{\substack{i<j,\ j-i\le 18 \\ t_{i:j}\in V}} \bigl(B(i) + (j-i)^2\bigr), \qquad B(0) = 0, \tag{3}$$

and keep the segmentation behind $B(L)$ if it has at least two words. Squaring the word length prefers a few long words to many short ones.

Picture-only slides. When the largest picture covers at least 45% of the page and the page has at most twelve words of real text, the picture is read with Tesseract [19]. Let I be the set of dark pixels and M the union of the OCR word boxes (padded by four pixels). The graphic ratio

$$\gamma = \frac{|I \setminus M|}{|I|} \tag{4}$$

is close to zero for a screenshot of bullet text and large for a diagram. The OCR text replaces the picture only if mean confidence is at least 62, there are at least twelve words, $\gamma \le 0.42$, at most 15% of the lines are symbol debris and the text does not look like a photographed table. Otherwise the picture is kept as a figure. This check matters for mathematics: an equation read by OCR ("`) = o where o, is unknown`") is worse than the picture of the equation.

Charts and diagrams. When most "bullets" of a slide are numbers with an average of under four words, they are axis ticks of a drawn chart; the region below the title is rendered as an image instead. A page with ten or more drawn shapes and almost no bullet glyphs is treated as a vector diagram in the same way.

PPTX input. A PowerPoint file is first rendered to PDF by LibreOffice in headless mode (cached by file hash) and then read as above. Reading shapes one by one with python-pptx lost slides that contained only pictures and produced titles such as "Slide 9"; on our Linear Regression deck the shape parser produced 23 slides from 63, while the PDF route produced 69 slides covering every source slide.

V. Content Decision Engine

For each slide the engine runs a set of detectors. Each detector d returns either nothing or a tuple $(f_d, v_d, c_d, D_d, r_d)$: a visual family, a variant, a confidence $c_d \in [0,1]$, the data the renderer needs and a human-readable reason. Examples are a *timeline* (three or more sentences with a date and an event that has words), a *process* (ordinal cues such as "first", "then",

numbered steps), a *comparison table* (parallel sentences about two or more subjects), a *definition* ("X is a …"), *stat tiles* (numbers with units and labels) and charts. Tables and section outlines are checked first because they are unambiguous. The remaining candidates $\mathcal{C}$ are filtered by exclusion rules E learned from the gold set (for example, a cycle suppresses a linear process, and a definition suppresses stat tiles), and the decision is

$$d^{*} = \arg\max_{d \in E(\mathcal{C})} c_{d}, \tag{5}$$

with the next three candidates kept as alternatives for the editor. When nothing structural is found the slide becomes *key-point cards*, which carry every sentence in full, so choosing a visual never removes text.

Card headings are generated from the sentence itself. A heading is the first three content words after removing discourse cues and pronouns; trailing function words, modal verbs, adverbs ending in "-ly" and anything after an unbalanced parenthesis are dropped, so a heading never ends mid-phrase ("Inductive bias (also" becomes "Inductive bias").

VI. Layout Without Broken Words

Every box is sized from its content, and two width models are used. For counting lines, a box of width W inches holds about $\lfloor 72W/(\bar{\rho}\, s) \rfloor$ characters per line at font size s, with an average advance ratio $\bar{\rho} = 0.52$ for regular and 0.575 for bold text. For single words, which must never be split, the width is measured glyph by glyph:

$$w(u, s) = \kappa \, \frac{s}{72} \sum_{ch \in u} \rho(ch), \tag{6}$$

where $\rho(ch)$ is the advance width of each printable character taken from the font metrics (separate tables for regular and bold) and $\kappa = 1.06$ is a safety factor for viewers that substitute a slightly wider font. A word is split into unbreakable pieces after hyphens and slashes, where PowerPoint itself may wrap. The number of lines is estimated by greedy wrapping, and the height of a block of n lines is $n \cdot 1.26\, s/72$. The font size chosen for a box of size $W \times H$ is

$$s^{*} = \max\{\, s \in \mathcal{G} : H(t, s, W) \le H \,\}, \tag{7}$$

where $\mathcal{G} = \{s_{\max}, s_{\max} - 0.5, \ldots, s_{\min}\}$ is the half-point grid, and $s_{\min} = 11\,\text{pt}$ for body text. Content that does not fit at $s_{\min}$ is moved to a continuation slide instead of being shrunk further. Separately, every paragraph is scaled by

$$\sigma = \max\Big(0.62,\ \min\big(1,\ \min_{u} \tfrac{W}{w(u,s)}\big)\Big), \tag{8}$$

where u ranges over the unbreakable pieces of the text. This guarantees that the longest word fits on one line, which is the error students notice first ("Homo-nym" split across two lines).

The layout audit opens the finished PPTX and flags, for each slide, text that overflows its box, words broken across lines, slides whose content covers too little of the area (sparse) and slides with an empty lower half.

Algorithm 1 Per-slide illustration with fallbacks

Require: slides S, key pool K, cache C
for each topic slide $s \in S$ (in parallel, 5 workers) **do**
  if s has a source figure **then**
    show it beside the content; **continue**
  end if
  $h \leftarrow \mathrm{hash}(title_s, text_s)$
  if $h \in C$ **then**
    $g \leftarrow C[h]$
  else
    for key k in round-robin order of live keys in K **do**
      $a \leftarrow$ ASK(k, prompt(s))
      if a failed with auth/quota error **then**
        mark k dead; **continue**
      end if
      $g \leftarrow$ VALIDATE(a); **break**
    end for
    $C[h] \leftarrow g$ ▷ also caches failures
  end if
  if $g \neq \varnothing$ **then**
    attach g as SVG and PNG
  else
    attach offline scene for s
  end if
end for

VII. Illustrations With Two Fallbacks

Every topic slide receives a picture, chosen in this order: (i) the source's own figure, shown beside the content; (ii) a diagram written by a language model; (iii) a scene from the offline illustrator. Quiz slides and slides that are already a full figure are left alone.

For (ii) the model receives the slide's title and text and is asked for one SVG figure in a 600×720 view box that shows the mechanism of the idea using the slide's own terms, with three to nine short labels and only the deck's colours. The answer is accepted only if it passes all of the following: it parses as XML; every element is one of a small set of drawing elements (no script, image, foreign object, link, CSS or filter); it has at least six shapes and at most sixteen text elements; and it renders with MuPDF. Labels that would run past the edge are scaled down using the same width estimate as in the layout stage. Accepted diagrams are cached by a hash of the slide text, served at a public URL, embedded as live SVG in the web deck and rasterised for PowerPoint, where the picture links to the full-size page.

Keys are used in a pool. Each key is paced on its own (one request per 4.2 s on the free tier) and requests are spread round-robin over the keys. A key that returns an authentication or quota error is marked dead for 15 minutes and the next key is tried; when all keys fail, the offline illustrator is used. Algorithm 1 summarises the stage.

TABLE I
VISUAL-FAMILY SELECTION ON THE GOLD SETS

Set	Slides	Strict acc.	Real visual	Families used
Development	38	97.4%	100%	24
Held-out	32	96.9%	100%	21
Blind	31	96.8%	100%	21

VIII. LANGUAGE-MODEL ASSISTANCE AND FAILOVER

With a key, slides also receive an assertion-style title (a full sentence with a verb, at most fifteen words), a short plain-language explanation for thin slides, one example and up to two key terms [12]. Each answer is checked against the slide: the title must share at least one content word with the slide and must not start with an imperative verb, and the explanation must share at least two. On a thin slide (fewer than 45 words) the explanation and example are added to the body as normal points. Requests go through a router over Groq, Gemini and NVIDIA endpoints with per-task preferences; a provider that fails four times in a row, or once with a dead-key error, is switched off for 15 minutes and the rule-based path continues, so a deck is never blocked by an API.

IX. EVALUATION

A. Visual selection

We labelled three gold sets of slide texts with the set of acceptable visual families: a development set (38 slides), a held-out set (32) and a blind set (31) written by a team member who had not seen the rules. A case counts as *strict* correct when the chosen family is acceptable, renders as a real visual and receives data. Table I shows the results. The single miss in each set was a slide where two families were defensible and the annotator had chosen only one. Deciding a slide took a median of 0.48 ms (95th percentile 0.82 ms) on a laptop CPU.

B. Real course decks

We ran the system without any API key on six decks from our courses: three NLP slide PDFs (Module 4, 36 pages; Module 5, 95 pages; Syntax analysis, 216 pages), the WSD lecture slides (44 pages) and two machine-learning PowerPoint files (28 and 63 slides). Table II reports the size of the rebuilt deck, total time from upload to finished files, fact retention and the layout audit. Fact retention is

$$R = \frac{|F_{\mathrm{src}} \cap F_{\mathrm{deck}}|}{|F_{\mathrm{src}}|}, \tag{9}$$

where F is the set of numbers with units, years, percentages and capitalised named terms found in the source and in the generated deck.

No slide in any deck was flagged for a broken word or for overflow. All flags were *sparse* or *empty bottom*: slides whose source content was a single short line. The PPTX decks are slower because LibreOffice renders them to PDF first (about 11 s for the 63-slide deck) and because they contain many full-page figures.

TABLE II
REBUILT DECKS WITHOUT AN API KEY

Deck	Pages	Slides	Time (s)	R	Flagged
NLP Module 4	36	47	9.8	100%	0%
NLP Module 5	95	100	21.2	99%	3%
NLP Syntax analysis	216	225	34.4	100%	3%
WSD slides	44	53	14.8	98%	4%
ML: Bias and variance (PPTX)	28	33	11.7	100%	6%
ML: Linear regression (PPTX)	63	69	44.0	100%	7%

C. Effect of the PPTX route

On the Linear Regression deck the previous shape-by-shape parser produced 23 slides from 63 source slides, with titles such as "Types of regression models Slide 9" and three duplicated "Advantages and Disadvantages" slides. With the PDF route the rebuilt deck had 69 slides, every source slide was represented, 22 slides showed the original diagrams and derivations as figures, and no title contained "Slide n".

D. Illustrations

Without a key the offline illustrator draws a scene on every topic slide. With the five-key pool, generating validated SVG diagrams for four test slides in parallel took 8.9 s in total, and in the full ML deck the diagrams were prepared in parallel with the other stages. Diagrams that failed validation (three of the first eleven, mostly because of disallowed elements or too little drawing) were replaced by offline scenes, so no slide was left without a picture.

X. DISCUSSION AND LIMITATIONS

The decision engine is rule-based by choice: its errors can be read and fixed, it costs nothing to run and it works in a classroom without internet. The price is coverage; slides whose meaning lies mainly in a picture still depend on the original figure. OCR remains the weakest part of the reader for photographed equations, which we now keep as pictures rather than text. Without a key, slides that are thin in the source stay thin, because the system does not invent content. We have not yet run a controlled study with students; the gold sets measure whether the visual is appropriate, not whether it improves learning.

XI. CONCLUSION

Learnova rebuilds existing lecture decks instead of generating new ones. By treating the teacher's content as fixed and making every structural decision with inspectable rules, it keeps 98–100% of the facts of real course decks, chooses an appropriate visual for about 97% of slides and never breaks a word across lines. Language models are used only where they help and can be checked—titles, explanations and diagrams—and the system keeps working when every key is exhausted. The code, gold sets and measurement scripts are available in the project repository.

ACKNOWLEDGMENT

We thank our project mentor and the faculty of the Department of Artificial Intelligence and Data Science, DJSCE, for their guidance and for the lecture material used in the evaluation.

REFERENCES

[1] Gamma Tech, "Gamma: AI-powered presentations," https://gamma.app, 2026, accessed Oct. 2026.

[2] Microsoft, "Copilot in PowerPoint," https://support.microsoft.com/copilot-powerpoint, 2026, accessed Oct. 2026.

[3] Google, "Gemini in Google Slides," https://workspace.google.com/products/slides/, 2026, accessed Oct. 2026.

[4] Canva, "Magic design for presentations," https://www.canva.com/magic-design/, 2026, accessed Oct. 2026.

[5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1906–1919.

[6] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[7] Y. Hu and X. Wan, "PPSGen: Learning-based presentation slides generation for academic papers," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 4, pp. 1085–1097, 2015.

[8] T.-J. Fu, W. Y. Wang, D. McDuff, and Y. Song, "DOC2PPT: Automatic presentation slides generation from scientific documents," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 1, 2022, pp. 634–642.

[9] E. Sun, Y. Hou, D. Wang, Y. Zhang, and N. X. R. Wang, "D2S: Document-to-slide generation via query-based text summarization," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, 2021, pp. 1405–1418.

[10] J. Mackinlay, "Automating the design of graphical presentations of relational information," *ACM Transactions on Graphics*, vol. 5, no. 2, pp. 110–141, 1986.

[11] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer, "Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 1, pp. 438–448, 2019.

[12] M. Alley, M. Schreiber, K. Ramsdell, and J. Muffo, "How the design of headlines in presentation slides affects audience retention," *Technical Communication*, vol. 53, no. 2, pp. 225–234, 2006.

[13] J. K. Garner and M. P. Alley, "How the design of presentation slides affects audience comprehension: A case for the assertion–evidence approach," *International Journal of Engineering Education*, vol. 29, no. 6, pp. 1564–1579, 2013.

[14] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge: Cambridge University Press, 2009.

[15] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[16] E. R. Tufte, *The Cognitive Style of PowerPoint*. Cheshire, CT: Graphics Press, 2003.

[17] D. E. Knuth and M. F. Plass, "Breaking paragraphs into lines," *Software: Practice and Experience*, vol. 11, no. 11, pp. 1119–1184, 1981.

[18] H. El Hattab, "reveal.js: The HTML presentation framework," https://revealjs.com, 2026, accessed Oct. 2026.

[19] R. Smith, "An overview of the Tesseract OCR engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.